

Практика по курсу «Машинное обучение»

Петруша Полина Георгиевна

Контакты и информация

- Материалы и папка для сдачи лабораторных

<https://cloud.etu.ru/s/KFnM4wt29gMaoq9>



- Информация и все вопросы по курсу – в ТГ чате

<https://t.me/+Pnbc6Uc2F7w5Yjhi>

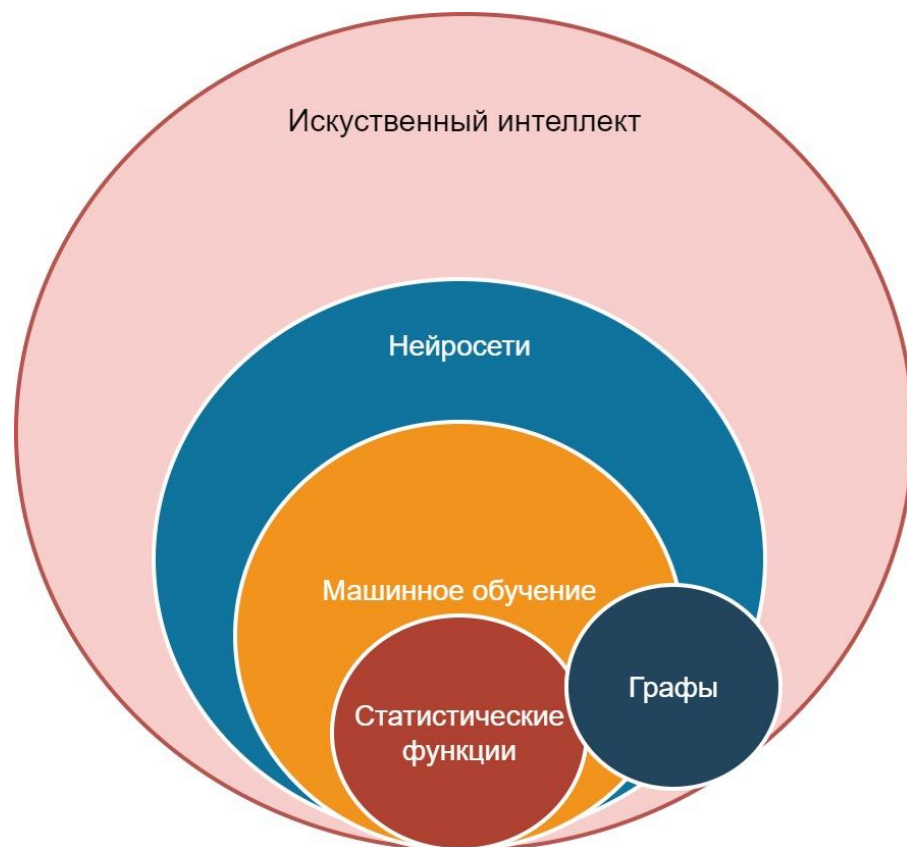


- Лично – 3117 (предварительно уточнив на месте ли я в ТГ)

Что надо будет сделать?

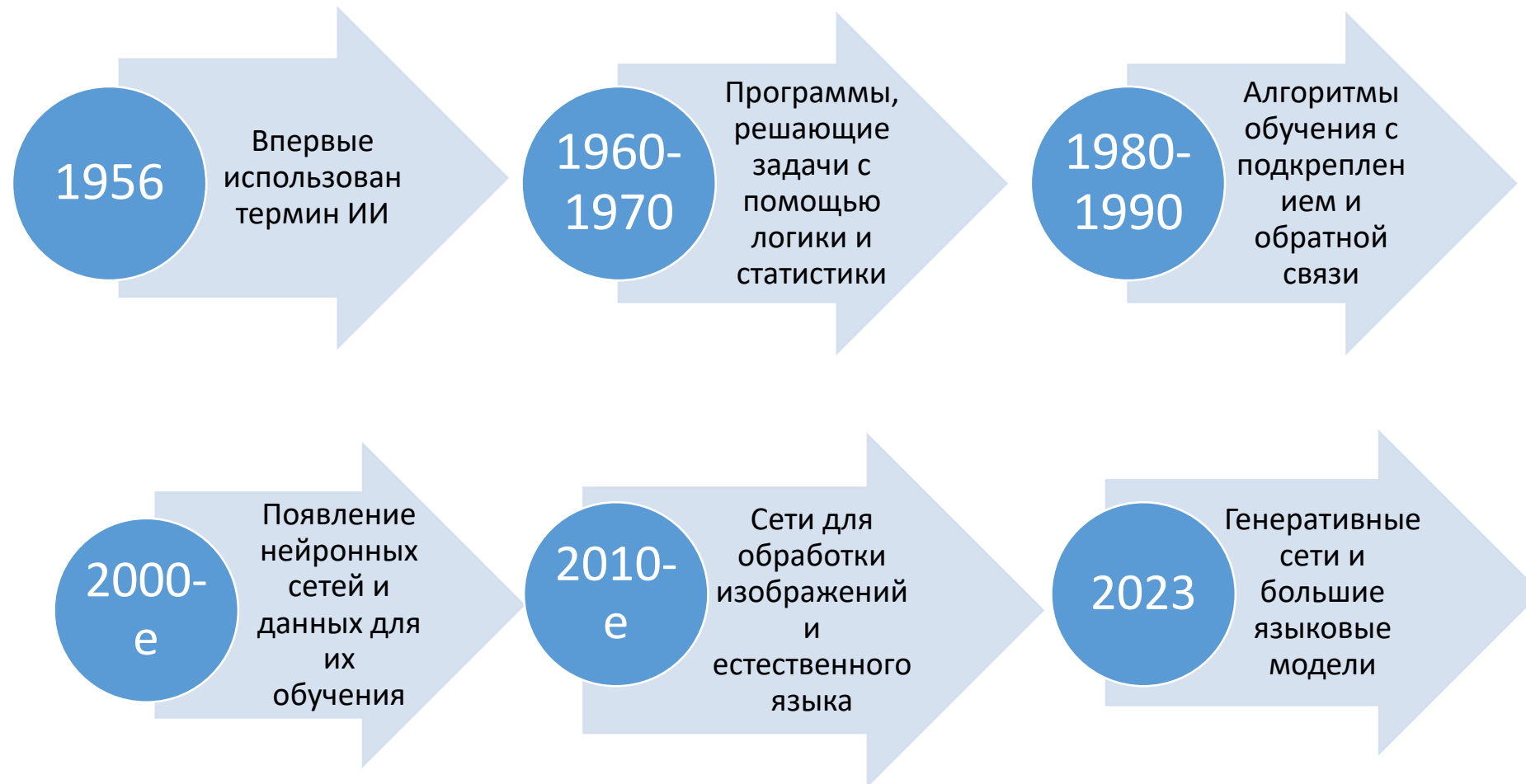
- 3 лабораторные работы
- 1 лабораторная работа = 20 баллов
- Срок на 1 лабораторную = 3 недели (дата начинается с дня выдачи информации по лабораторной)
- 60 баллов = 5 за экзамен «автоматом»
- 20 баллов допуск к экзамену в принципе (если хотите повысить оценку)
- Форма сдачи – отчет со ссылкой на kaggle notebook или google colab с кодом + пояснения и комментарии

Машинное обучение, нейросети, графы, ИИ



- **Статистические функции** — математические инструменты, позволяющие анализировать, интерпретировать и делать выводы на основе наборов данных
- **Машинное обучение** – применение статистических функций, позволяющих сначала их «обучать» на наборе данных, а потом применить эти алгоритмы на аналогичных
- **Нейросети** - специфическая архитектура алгоритмов машинного обучения. Состоят из слоев взаимосвязанных нейронов, каждый из которых обрабатывает определенный аспект входных данных.
- **Анализ графов** - набор методов, направленных на изучение связей между сущностями для выявления неочевидных зависимостей.
- **Искусственный интеллект** – совокупность всех алгоритмов, которые позволяют обрабатывать большие объемы данных, предсказывать события или принимать решения.

История ИИ



Машинное обучение.

Что это?

- **Машинное обучение** - это подобласть искусственного интеллекта, которая позволяет компьютерным системам обучаться на основе статистических алгоритмов, опыта и данных.
- С помощью машинного обучения компьютерные программы могут самостоятельно улучшать свою производительность и адаптироваться к новым задачам.
- **Алгоритмы машинного обучения** - это набор методов и моделей, которые позволяют компьютерным системам самостоятельно извлекать знания из данных и принимать решения на основе полученной информации.
- В рамках алгоритмов машинного обучения, **модели обучаются на тренировочных данных** и затем могут применять полученные знания для классификации, регрессии, кластеризации и других задач.

Лабораторная №1

- Служит для получения и закрепления навыков предобработки данных для дальнейшего применения методов машинного обучения для решения задач.
- Студент самостоятельно выбирает набор **сырых (не разобранных)** данных на сайте [kaggle.com](https://www.kaggle.com)
- **Требования:**
 - Число столбцов признаков – не менее 10;
 - Число записей – не менее 10000;
 - Набор данных имеет пропуски.
- **Задачи:**
 - Очистка данных (удаление пропусков, нормализация, удаление дубликатов)
 - Визуализация значимых признаков
 - диаграммы рассеяния
 - ящики с усами
 - гистограммы
 - Корреляция данных (матрица корреляций)

Подгрузка библиотек и данных

1. Загрузка библиотек

<https://www.kaggle.com/code/alexisbcook/scatter-plots?scriptVersionId=126573698&cellId=2>

```
import pandas as pd
pd.plotting.register_matplotlib_converters()
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
print("Setup Complete")
```

2. Загрузка датасета (csv, txt)

<https://www.kaggle.com/code/alexisbcook/scatter-plots?scriptVersionId=126573698&cellId=4>

```
# Path of the file to read
insurance_filepath = "../input/insurance.csv"
# Read the file into a variable insurance_data
insurance_data = pd.read_csv(insurance_filepath)
```

Очистка данных

1. Удаление пропусков

```
data.isnull().sum()
```

```
Unnamed: 0    0
```

```
price_doc    0
```

```
full_sq     0
```

```
life_sq     0
```

```
num_room    0
```

```
kitch_sq    0
```

```
dtype: int64
```

Если данные пропущены, то пропуски заполняются медианными данными по соответствующему столбцу

```
for t in data['income_type'].unique():
```

```
    data.loc[(data['income_type'] == t) & (data['days_employed'].isna()), 'days_employed'] = \
```

```
    data.loc[(data['income_type'] == t), 'days_employed'].median()
```

Тьюториал

<https://www.kaggle.com/code/andreyveld/educational-project-bank>

Очистка данных

2. Нормализация

По строке

```
normalized_arr = preprocessing.normalize([x_array])  
print(normalized_arr)
```

По столбцу

```
import pandas as pd  
housing = pd.read_csv("/content/sample_data/california_housing_train.csv")
```

Выбираем столбец и преобразуем его в массив

```
from sklearn import preprocessing  
x_array = np.array(housing['total_bedrooms'])  
normalized_arr = preprocessing.normalize([x_array])  
print(normalized_arr)
```

Очистка данных

3. Удаление дубликатов

```
import pandas as pd

# Создание DataFrame
df = pd.DataFrame({'A': [1, 1, 2], 'B': [2, 2, 2], 'C': [3, 3, 3]})

# Удаляем дублированные строки
df_unique = df.drop_duplicates()

print(df_unique)

# Определяем правило – для каждого набора повторяющихся значений первое вхождение имеет значение False, а все остальные - True.>>> df.duplicated()

# 0    False
# 1     True
# 2    False
# 3    False
# 4    False

# dtype: bool
```

что бы посмотреть DataFrame без дубликатов, необходимо в исходный DataFrame передать отрицание `~df.duplicated()` или использовать метод `DataFrame.drop_duplicates()`:

Тьюториал

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.drop_duplicates.html

https://pandas.pydata.org/docs/user_guide/duplicates.html

Визуализация Диаграммы

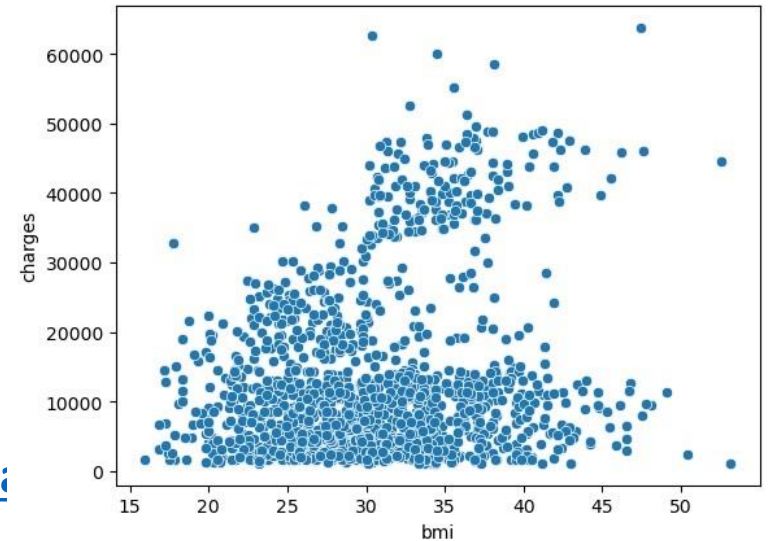
Диаграмма рассеяния

```
plt.figure(figsize=(10,7)) # Specify size of the chart  
plt.scatter('attack', 'defense', data=data[data.is_legendary==1], marker = 'x', c = 'magenta')  
plt.scatter('attack', 'defense', data=data[data.is_legendary==0], marker = 'o', c = 'blue')  
plt.legend(('Yes', 'No'), title='Is legendary?')  
plt.show()
```

Тьюториалы

https://matplotlib.org/stable/api/as_gen/matplotlib.pyplot.scatter.html

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.scatter.html>



Визуализация Диаграммы

Ящики с усами

```
import plotly.graph_objects as go
import numpy as np
np.random.seed(1)
```

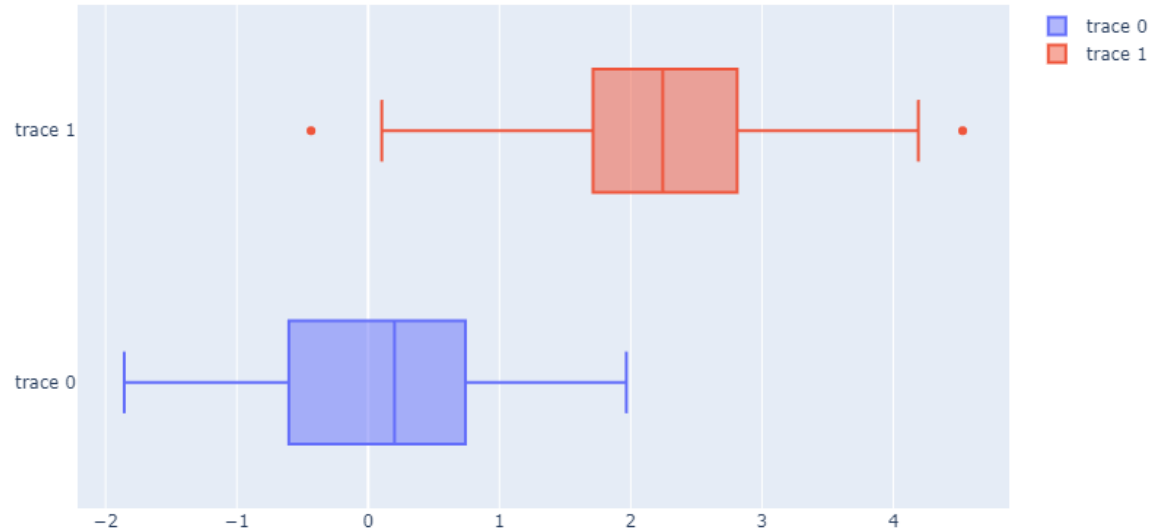
```
y0 = np.random.randn(50) - 1
y1 = np.random.randn(50) + 1
```

```
fig = go.Figure()
fig.add_trace(go.Box(y=y0))
fig.add_trace(go.Box(y=y1))
fig.show()
```

Ссылки на примеры и тьюториалы

<https://www.kaggle.com/code/babyoda/improved-box-plot-visualization>

<https://www.kaggle.com/code/kaushiksuresh147/box-plot-python-implementation>



Визуализация Диаграммы

Гистограммы

<https://www.kaggle.com/code/alexisbcook/scatter-plots?scriptVersionId=126573698&cellId=8>

In

```
# Histogram
```

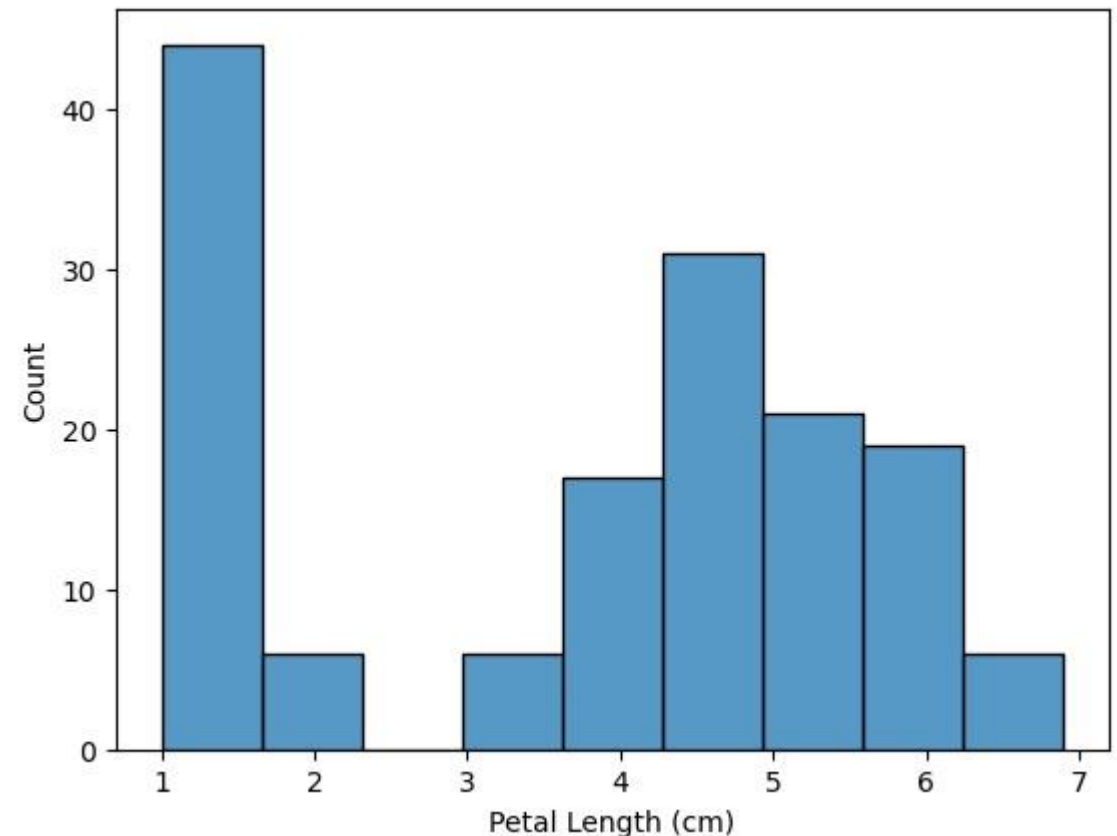
```
sns.histplot(iris_data['Petal Length (cm)'])
```

Out

```
<AxesSubplot:xlabel='Petal Length (cm)', ylabel='Count'>
```

Ссылки на примеры и тьюториалы

<https://www.kaggle.com/code/alexisbcook/distributions>



Статистический анализ

Библиотека статистических инструментов <https://docs.scipy.org/doc/scipy/reference/stats.html>

Тьюториал <https://www.kaggle.com/code/emstrakhov/lesson-2-correlation-analysis>

Первичная матрица

```
df['age_years'] = df['age'] / 365.25 # возраст в годах  
  
numeric = ['age_years', 'height', 'weight', 'ap_hi', 'ap_lo', 'cholesterol', 'gluc']  
  
sns.pairplot(df[numeric]);
```

Удаление выбросов

```
sns.boxplot(df['age_years']);  
  
def outliers_indices(feature):  
  
    Будем считать выбросами все точки, выходящие за пределы трёх сигм.  
  
    mid = df[feature].mean()  
    sigma = df[feature].std()  
  
    return df[(df[feature] < mid - 3*sigma) | (df[feature] > mid + 3*sigma)].index  
  
wrong_height = outliers_indices('height')  
wrong_weight = outliers_indices('weight')  
wrong_hi = outliers_indices('ap_hi')  
wrong_lo = outliers_indices('ap_lo')  
  
out = set(wrong_height) | set(wrong_weight) | set(wrong_hi) | set(wrong_lo)  
  
print(len(out))  
  
df.drop(out, inplace=True)
```

Матрица без выбросов

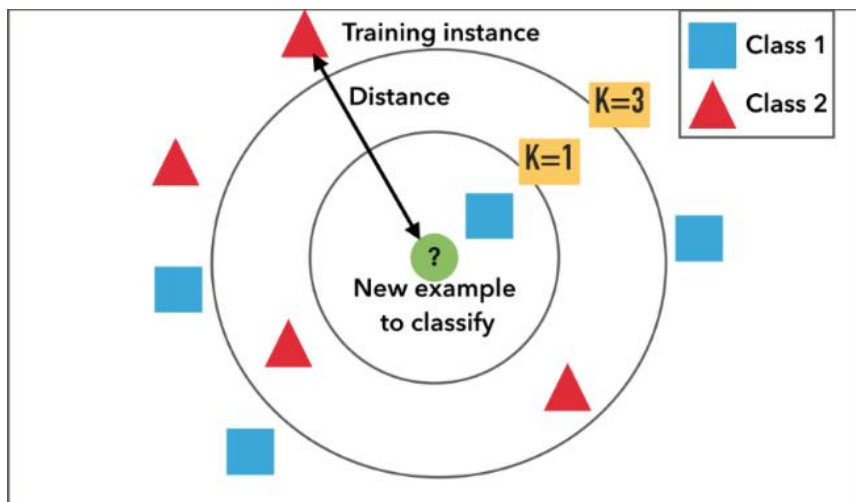
```
sns.pairplot(df[numeric]);  
  
sns.heatmap(df[numeric].corr(method='spearman'));
```

Лабораторная №2

- Служит для получения и закрепления навыков предобработки данных и применения методов машинного обучения для решения задач классификации.
- Набор данных берется из лабораторной работы №1.
- Этапы
 - Обучение моделей и подбор параметров (с использованием Grid Search):
 - К-ближайших соседей (KNN)
 - Машина опорных векторов (SVM)
 - Дерево решений ИЛИ Случайный лес
 - Оценка моделей
 - Оценка качества прогноза (precision/recall/f1-score/ROC-AUC)
 - Визуализация дерева решений ИЛИ Визуализация Feature Importance для случайного леса

KNN (метод ближайших соседей)

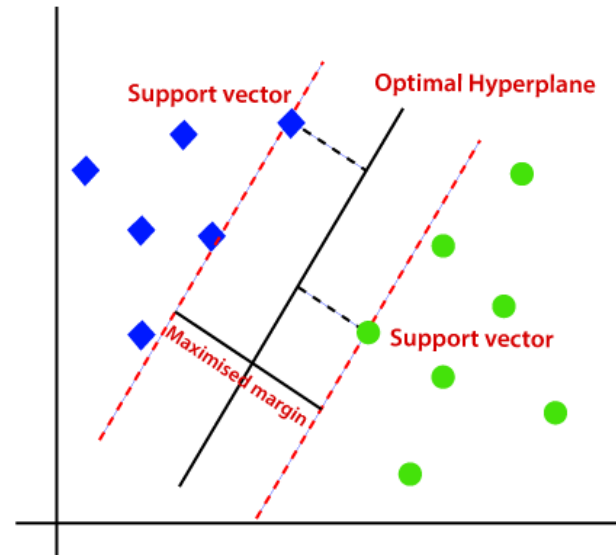
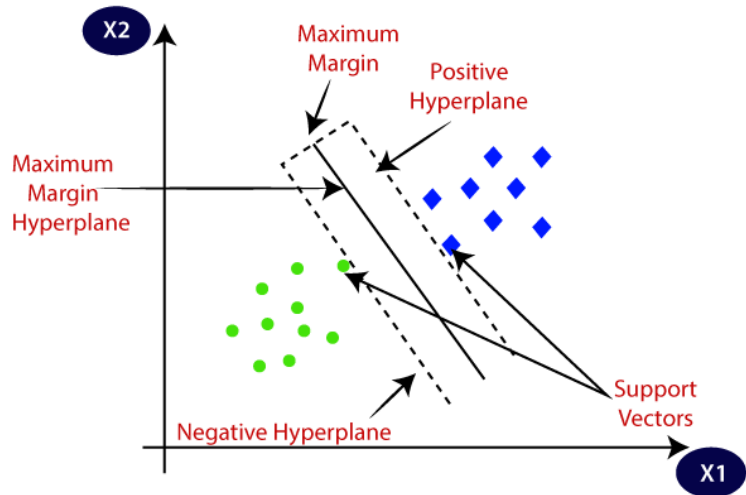
- Метод k ближайших соседей (англ. kNN — k Nearest Neighbours) — Для повышения надёжности классификации объект относится к тому классу, которому принадлежит большинство из его соседей — k ближайших к нему объектов обучающей выборки x_i .
- В задачах с двумя классами число соседей берут нечётным, чтобы не возникало ситуаций неоднозначности, когда одинаковое число соседей принадлежат разным классам.



- <https://www.kaggle.com/code/just4jcgeorge/k-nearest-neighbour-algorithm>
- <https://www.kaggle.com/code/amolbhivarkar/knn-for-classification-using-scikit-learn>
- <https://www.kaggle.com/code/prashant111/knn-classifier-tutorial>

Метод опорных векторов (англ. support vector machine, SVM)

- Метод опорных векторов (англ. support vector machine, SVM) — один из наиболее популярных методов обучения, который применяется для решения задач классификации и регрессии.
- Основная идея метода заключается в построении гиперплоскости, разделяющей объекты выборки оптимальным способом.
- Алгоритм работает в предположении, что чем больше расстояние (зазор) между разделяющей гиперплоскостью и объектами разделяемых классов, тем меньше будет средняя ошибка классификатора.



Метод опорных векторов (англ. support vector machine, SVM)

- <https://www.kaggle.com/code/satishgunjal/tutorial-support-vector-machines>
- <https://www.kaggle.com/code/rajeev102001/svm-support-vector-machine-for-classification>
- <https://www.kaggle.com/code/azzion/svm-for-beginners-tutorial>
- <https://www.kaggle.com/discussions/general/364536>
- <https://www.kaggle.com/code/ihsncnkz/classification-with-support-vector-machine-svm>

Grid Search

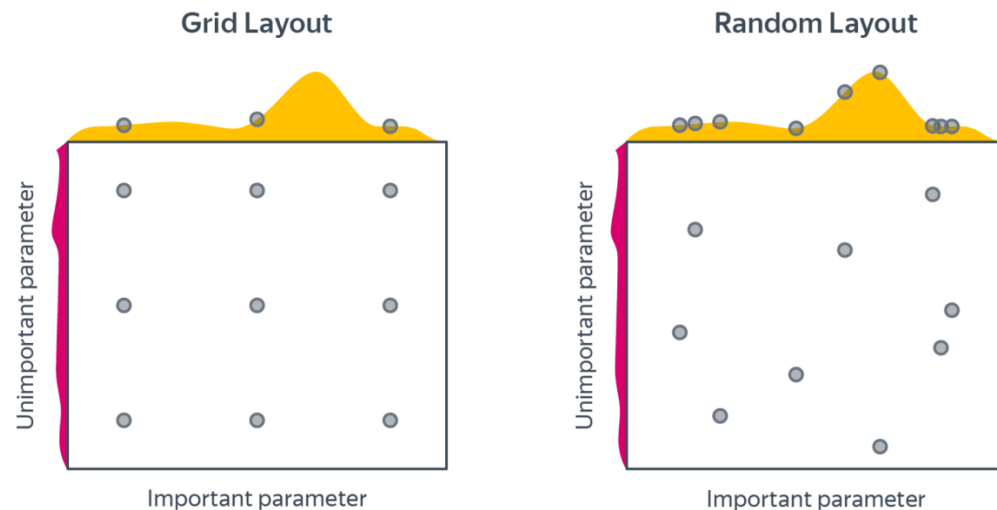
- Основан на принципе [Random Forest](#)
- Предполагает задание наборов значений для каждого гиперпараметра, которые затем "перебираются" систематически для нахождения наилучшей комбинации.
- Для каждой комбинации гиперпараметров производится обучение модели и оценка её производительности на валидационных данных
- Гарантирует полный перебор всех заданных комбинаций гиперпараметров, что может помочь найти наилучшие значения.
- Может быть очень ресурсозатратным, особенно при большом числе гиперпараметров и значений
- Различие между гиперпараметрами и параметрами модели – гиперпараметры задаются вручную до начала обучения и определяют характеристики всего процесса обучения, в то время как параметры модели вычисляются в процессе обучения на основе данных и оптимизируются для достижения наилучшей производительности.
- Гиперпараметры-настройки инструмента, с помощью которого вы создаете модель, а параметры модели - это результат работы этого инструмента на конкретных данных.

Grid Search

- Примеры:
 - для метода ближайших соседей можно, например, перебирать по сетке число соседей (например, от 1 до 20) и метрику, по которой будет измеряться расстояние между объектами выборки (евклидова, манхэттенская...);
 - для решающих деревьев можно перебирать по сетке сочетания значений максимальной глубины дерева и различные критерии ветвления (критерий Джини, энтропийный критерий).
- Перебор некоторых значений гиперпараметров можно вести по логарифмической шкале, так как это позволяет быстрее определить правильный порядок параметра и в то же время значительно уменьшить время поиска.
- Сразу же видно естественное ограничение данного метода: если комбинаций параметров слишком много либо каждое обучение / тест длится долго, алгоритм не завершится за разумное время.

Grid Search + Random Forest

- **Количество деревьев ($n_estimators$):** Определяет, сколько деревьев будет построено в ансамбле.
- **Максимальная глубина деревьев (max_depth):** Ограничивает глубину каждого дерева, предотвращая переобучение.
- **Минимальное количество объектов в листе ($min_samples_leaf$):** Задаёт минимальное количество объектов, которые должны находиться в каждом листе дерева.
- **Максимальное количество признаков для разбиения ($max_features$):** Определяет максимальное количество признаков, которые рассматриваются при каждом разбиении узла.



Grid Search

- <https://www.kaggle.com/code/carlmcbrideellis/hyperparameter-grid-search-simple-example>
- <https://www.kaggle.com/code/pythonafroz/step-by-step-guide-to-gridsearchcv>
- <https://www.kaggle.com/code/sanikamal/grid-search-explained>
- <https://www.kaggle.com/code/willkoehrsen/intro-to-model-tuning-grid-and-random-search>

Оценка моделей

- Оценка качества прогноза:
 - Precision
 - Recall
 - f1-score
 - ROC-AUC
- Визуализация дерева решений ИЛИ Визуализация Feature Importance для случайного леса

Оценка моделей.

- Общая точность прогнозов (**accuracy**) -показывает долю правильных классификаций ко всей выборке,
- Полнота(**recall**) -показывает отношение верно классифицированных объектов класса к общему числу элементов этого класса
- Точность прогнозов о принадлежности к классу (**precision**) - показывает долю верно классифицированных объектов среди всех объектов, которые к этому классу отнес классификатор
- **F1-мера** - среднее гармоническое между precision и recall

[Ссылка](#)

Оценка моделей.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score#
```

Пример вычисления различных метрик

```
y_pred= model.predict(X_val)
accuracy= accuracy_score(y_val, y_pred)
precision= precision_score(y_val, y_pred)
recall= recall_score(y_val, y_pred)
f1 = f1_score(y_val, y_pred)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
```

<https://scikit-learn.ru/3-3-metrics-and-scoring-quantifying-the-quality-of-predictions/>

Оценка моделей. Дерево решений

Дерево решений — это один из алгоритмов машинного обучения. Алгоритм основан на правиле: «Если <условие>, то <ожидаемый результат>

В машинном обучении, статистике и анализе данных с помощью метода дерева решений можно строить прогнозы, описывать данные, разделять их на группы и находить зависимости между ними.

Простая и популярная задача — это бинарная классификация. То есть разделение множества элементов на две группы, где

1 — успех, да, ответ верный,

0 — неудача, нет, ответ неверный

Оценка моделей. Дерево решений

- Простейшая модель дерева решений

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Создание модели
```

```
model = DecisionTreeClassifier()
```

```
# Обучение модели
```

```
model.fit(X_train, y_train)
```

```
# Прогнозирование
```

```
predictions = model.predict(X_test)
```

- С использованием TensorFlow Decision Forests

TensorFlow Decision Forests (TF-DF) — это библиотека для обучения, запуска и интерпретации моделей леса решений (например, случайных лесов, деревьев с градиентным усилением) в TensorFlow. TF-DF поддерживает классификацию, регрессию, ранжирование и повышение.

https://keras.io/examples/structured_data/classification_with_tfdf/

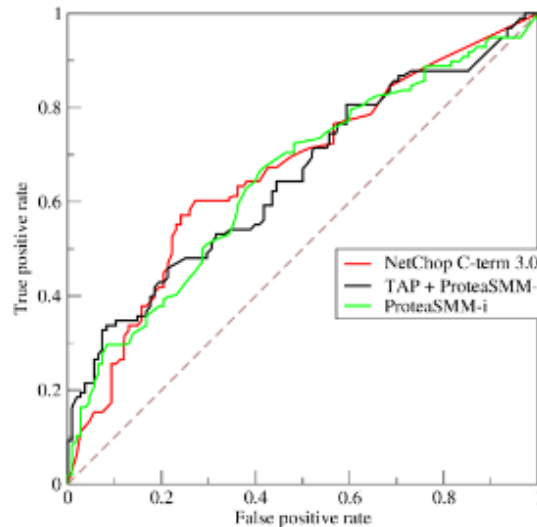
https://www.tensorflow.org/decision_forests/tutorials/beginner_colab?hl=ru

Оценка моделей. ROC-AUC

ROC-кривая (англ. receiver operating characteristic, рабочая характеристика приёмника) — график, позволяющий оценить качество бинарной классификации, отображает соотношение между **долей объектов от общего количества носителей признака, верно классифицированных как несущие признак** (англ. true positive rate, **TPR**, называемой чувствительностью алгоритма классификации), и **долей объектов от общего количества объектов, не несущих признака, ошибочно классифицированных как несущие признак** (англ. false positive rate, **FPR**, величина $1 - \text{FPR}$ называется специфичностью алгоритма классификации) при варьировании порога решающего правила.

Оценка моделей. ROC-AUC

Кривая ROC – это график, который иллюстрирует производительность классификационной модели при всех возможных порогах классификации.



Ось X данного графика представляет собой FPR, т.е ложноположительную частоту, а ось Y — TPR, т.е истинноположительную частоту. TPR также известен как Recall, и определяется как доля правильно классифицированных положительных результатов относительно всех положительных результатов в данных:

$$\text{TPR} = \frac{\text{TP (true positive, истинно положительные результаты)}}{\text{TP + FN (false negative, ложноотрицательные результаты)}}$$

$$\text{TP + FN (false negative, ложноотрицательные результаты)}$$

FPR определяет долю ошибочно классифицированных отрицательных результатов относительно всех отрицательных результатов

$$\text{FPR} = \frac{\text{FP (false negative)}}{\text{TN (true negative) + FP (false positive)}}$$

Оценка моделей. ROC-AUC

Показатель AUC (Area Under the ROC Curve) — это мера, которая позволяет суммировать производительность модели одним числом, измеряя площадь под кривой ROC.

AUC колеблется от 0 до 1, где более высокое значение AUC указывает на более высокую производительность модели.

AUC равный 0.5 указывает на отсутствие дискриминационной способности модели, тогда как AUC равный 1.0 означает идеальное различие классов.

Оценка моделей. ROC-AUC

инициализация и обучение модели

```
log_regression = LogisticRegression()
```

```
log_regression.fit(X_train, y_train)
```

предсказание вероятностей

```
y_pred_proba = log_regression.predict_proba(X_test)[: ,1]
```

расчет значений FPR, TPR

```
fpr, tpr, _ = metrics.roc_curve(y_test, y_pred_proba)
```

построение ROC кривой

```
plt.plot(fpr, tpr)
```

```
plt.ylabel('True Positive Rate')
```

```
plt.xlabel('False Positive Rate')
```

```
plt.show()
```

#оценка качества модели - расчет площади под ROC

```
auc = metrics.roc_auc_score(y_test, y_pred_proba)
```

```
print("AUC: %.3f" % auc)
```

Оценка моделей. ROC-AUC

инициализация и обучение модели

```
log_regression = LogisticRegression()
```

```
log_regression.fit(X_train, y_train)
```

предсказание вероятностей

```
y_pred_proba = log_regression.predict_proba(X_test)[: ,1]
```

расчет значений FPR, TPR

```
fpr, tpr, _ = metrics.roc_curve(y_test, y_pred_proba)
```

построение ROC кривой

```
plt.plot(fpr, tpr)
```

```
plt.ylabel('True Positive Rate')
```

```
plt.xlabel('False Positive Rate')
```

```
plt.show()
```

#оценка качества модели - расчет площади под ROC

```
auc = metrics.roc_auc_score(y_test, y_pred_proba)
```

```
print("AUC: %.3f" % auc)
```

Лабораторная №3

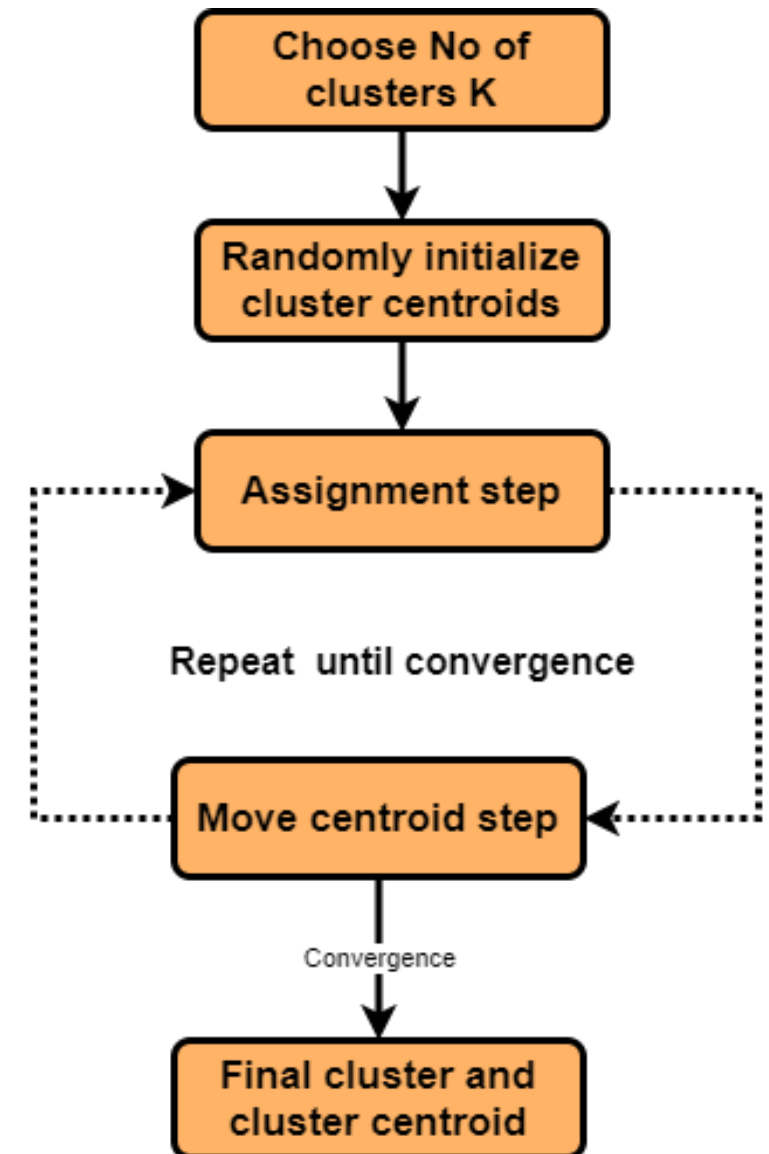
- Служит для получения и закрепления навыков предобработки данных и применения методов машинного обучения для решения задач кластеризации.
- Этапы
 1. Обучение моделей и подбор параметров (где применимо):
 - метод K-средних
 - Иерархическая кластеризация
 - DBSCAN
 - Линейная регрессия
 - LASSO
 - Ридж-регрессия
 2. Оценка моделей
 - а. Экспертная оценка
 - б. Сравнение разбиения на классы с помощью кластеризации
 - с. Визуализация предсказанных значений

Метод k-средних

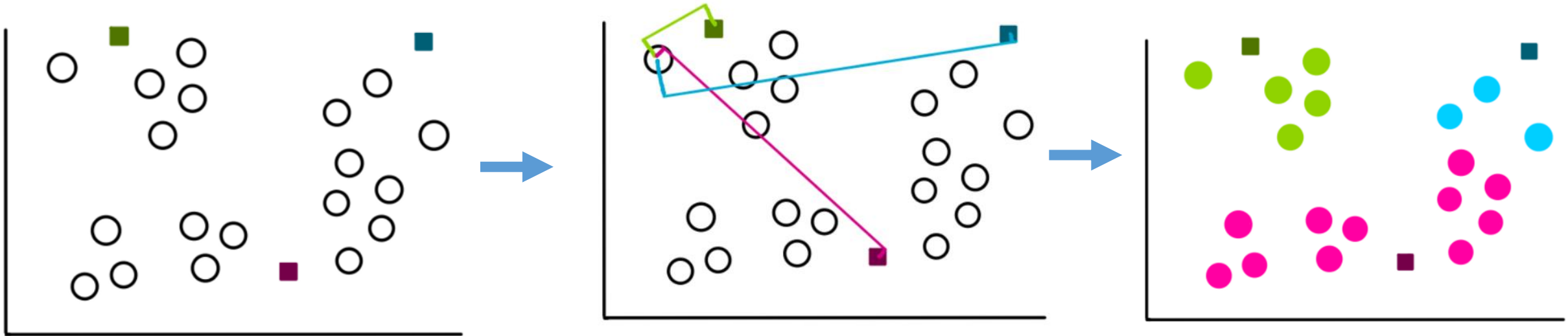
- Один из методов кластеризации. **Кластеризация** — это разделение множества входных векторов на группы (кластеры) по степени «схожести» друг на друга. Представители одного кластера должны быть схожи друг с другом больше, чем с представителями других кластеров по какому-либо критерию
- **Зачем он в ML/DM?**
 - Легче выделить группы схожих объектов, изучить их особенности и построить для каждой группы отдельную модель, чем создавать одну общую модель для всех данных
 - Сжатие данных
 - Обнаружение шумов
- Метод k-средних – это итеративный алгоритм, основанный на минимизации суммарного квадратичного отклонения точек кластеров от центров этих кластеров

Метод k-средних

- Заданное изначально множество разбивается на заранее известное количество кластеров k .
- При каждой итерации алгоритм стремится минимизировать среднеквадратичное отклонение точек каждого кластера. Данная идея реализуется благодаря переычислению на каждой итерации центра масс для каждого кластера, полученного на предыдущем шаге.
- После переычисления все элементы множества вновь разбиваются на кластеры, относя себя к тому, центр которого оказался ближе.
- Данные действия завершаются, если после переычисления центра масс кластера он совпал с центром масс кластера, полученным на предыдущей итерации



Метод k-средних



Метод k-средних

Проблемы

- не точный
- необходимо заранее знать количество кластеров. Можно опираться на какой-то закон - например, нормального распределения. После этого выполнялся классический алгоритм k-means, который давал более точные результаты.
- алгоритм очень чувствителен к выбору начальных центров кластеров. Классический вариант подразумевает случайный выбор кластеров, что очень часто являлось источником погрешности.
- не справляется с задачей, когда объект принадлежит к разным кластерам в равной степени или не принадлежит ни одному.
- Тьюториал <https://www.kaggle.com/code/satishgunjal/tutorial-k-means-clustering>
- Пошаговая инструкция <https://www.kaggle.com/code/shrutimechlearn/step-by-step-kmeans-explained-in-detail>

Иерархическая кластеризация

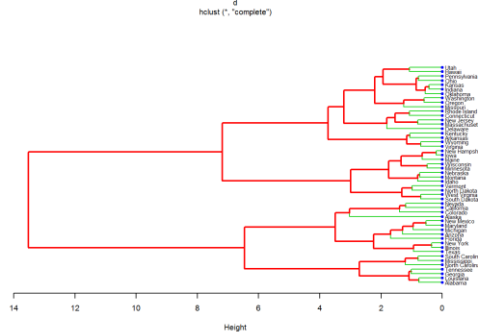
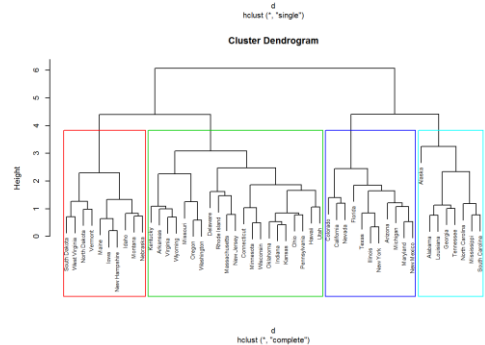
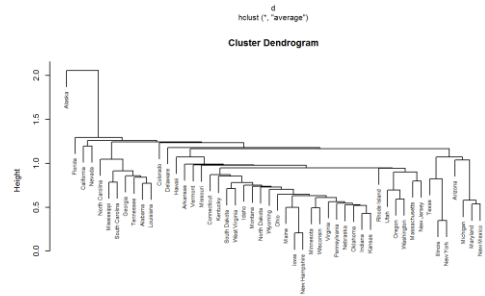
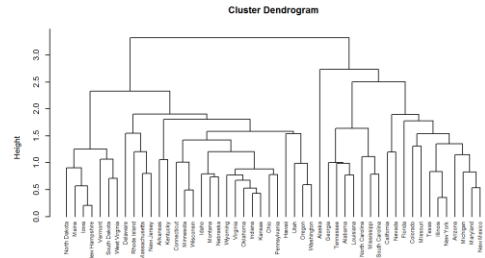
- Дерево строится
 - **от листьев к корню**
(агломеративные процедуры, где начальным является разбиение, состоящее из n одноэлементных классов, а конечным – из одного класса)
 - **от корней к листьям**
(дивизивные процедуры, где начальный класс мы бьем на n одноэлементных классов)



Иерархическая кластеризация(агломеративная)

- Инициализируем наше множество кластеров, каждая точка считается своим кластером. То есть для выборки размера N у нас на первой итерации будет N кластеров.
- Также входным параметром алгоритму подается метрика расстояния между двумя кластерами. Самой популярной метрикой является расстояние Уорда
- На каждой итерации мы объединяем два кластера в один. Объединяющиеся кластеры выбираются в соответствии с наименьшим расстоянием Уорда. То есть в соответствии с выбранным нами расстоянием эти два кластера будут наиболее похожи, поэтому объединяются
- Предыдущий шаг повторяется вплоть до объединения всех точек один кластер
- В результате в данном подходе мы можем выбрать любое количество кластеров после завершения процедуры, просто остановив на нужном нам шаге. К тому же данный алгоритм гораздо менее чувствителен к выбору метрики между точками, тогда как другие алгоритмы сильно зависят от этого.
- Для визуализации иерархической кластеризации удобно строить дендрограммы, в которых разница между уровнями равна выбранному расстоянию объединяющихся на данном этапе кластеров.

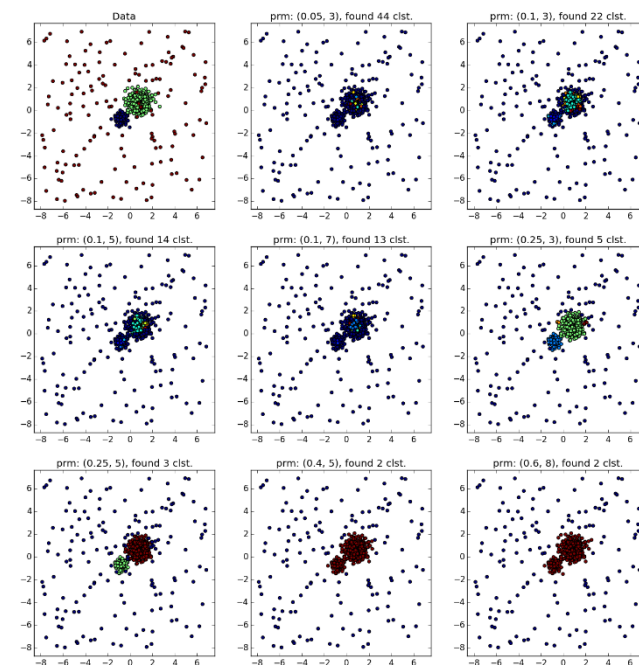
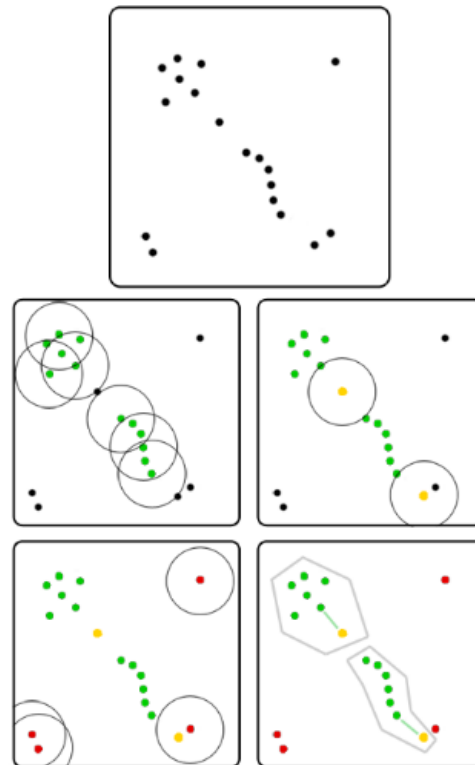
Иерархическая кластеризация



- Тьюториал <https://www.kaggle.com/code/vipulgandhi/hierarchical-clustering-explanation>
- Пример с объяснением <https://www.kaggle.com/code/arinderkm/hierarchical-clustering-mall-customers-dataset>

DBSCAN

- DBSCAN (Density-based spatial clustering of applications with noise, плотностной алгоритм пространственной кластеризации с присутствием шума) оперирует плотностью данных.
- На вход
 - матрица близости/расстояний
 - радиус окрестности
 - количество соседей



DBSCAN

- Плюсы
- работает на плотных, хорошо отделённых друг от друга кластерах вне зависимости от формы
- лучше всех неспециализированных алгоритмов обнаруживает кластеры малой размерности.
- Тьюториал <https://www.kaggle.com/code/meetnagadia/dbscan-clustering>
- Пример <https://www.kaggle.com/code/umairaslam/dbscan-clustering>

Линейная регрессия

Линейная регрессия (Linear regression) — один из простейших алгоритмов машинного обучения, описывающий зависимость целевой переменной от признака в виде линейной функции

Простая (парная) регрессия

$$y = kx + b$$

Множественная линейная регрессия

$$f_{wb}(x) = w_0x_0 + w_1x_1 + \dots + w_nx_n + b = w^*x + b$$

где b — смещение модели,

w — вектор её весов,

x — вектор признаков одного обучающего образца.

Линейная регрессия

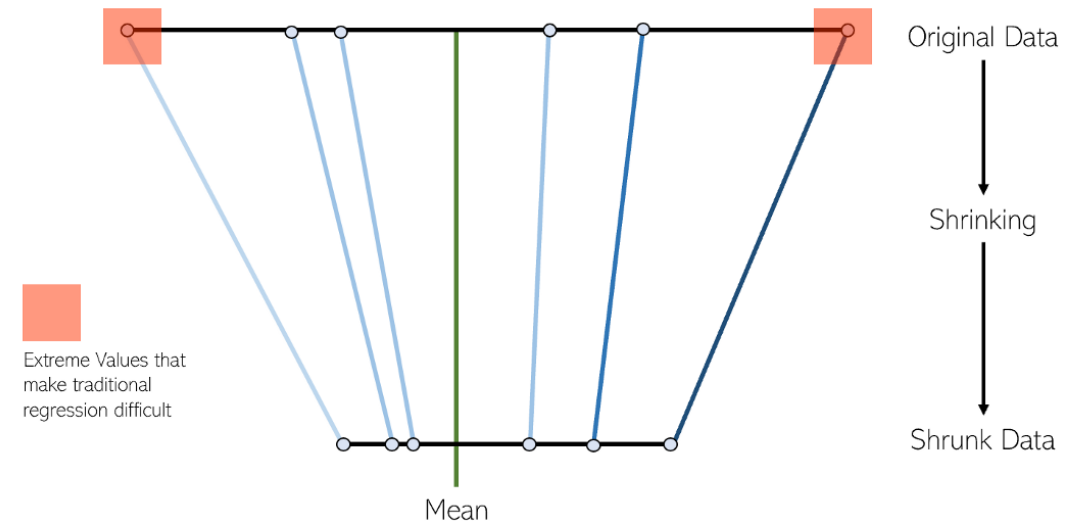
- Тьюториал с математикой и подробным описанием работы с kaggle <https://www.kaggle.com/code/sudhirnl7/linear-regression-tutorial>
- Тьюториал с предсказаниями продаж <https://www.kaggle.com/code/ashydv/sales-prediction-simple-linear-regression>
- И еще один тьюториал <https://www.kaggle.com/code/sarbjit87/linear-regression-tutorial-with-example>

LASSO

- Метод регрессии лассо (LASSO, Least Absolute Shrinkage and Selection Operator) — это вариация линейной регрессии, специально адаптированная для данных, которые **демонстрируют сильную мультиколлинеарность (то есть сильную корреляцию признаков друг с другом)**.
- Она автоматизирует части выбора модели:
 - выбор переменных
 - исключение параметров
- LASSO использует сжатие коэффициентов (shrinkage), то есть процесс, в котором значения данных приближаются к центральной точке (например среднему значению).
- Плюсы:
 - Более точные и стабильные оценки истинных параметров.
 - Уменьшение ошибок выборки и отсутствия выборки.
 - Сглаживание пространственных флуктуации

LASSO

- Позволяет уменьшить сложность данных так, чтобы их можно было обрабатывать простыми методами регрессии, искривляя пространство, на котором они лежат.
- Автоматически помогает устранить или исказить сильно коррелированные и избыточные функции в методе с низкой дисперсией.
- Используется регуляризация L1, то есть взвешивает ошибки по их абсолютному значению, а не регуляризацию L2, которая взвешивает ошибки по их квадрату, чтобы сильнее наказывать за более значительные ошибки.
- Это приводит к более разреженным моделям с меньшим количеством коэффициентов, поскольку некоторые из них становятся нулевыми и исключаются



LASSO

```
from sklearn.linear_model import LassoCV  
model = LassoCV()  
model.fit(X_train, y_train)
```

Тьюториалы

База <https://www.kaggle.com/code/deepakkumargunjetti/introduction-to-lasso-regression>

Предсказание с использованием LASSO <https://www.kaggle.com/code/acasalan/predict-bank-turnover-lasso-regression>

Гребневая (Риндж) регрессия

- Похожа на регрессию LASSO - применяет сжатие.
- Также хорошо подходит для наборов данных с большим количеством признаков, которые не являются независимыми друг от друга (коллинеарность).
- Использует регуляризацию L2, то есть ни один из коэффициентов не становится нулевым, как это происходит в регрессии LASSO. Вместо этого коэффициенты всё больше приближаются к нулю, но не имеют большого стимула достичь его из-за природы регуляризации
- Гребневая регрессия лучше подходит в ситуации, когда мы хотим сделать приоритетными большое количество переменных, каждая из которых имеет небольшой эффект.

Гребневая (Риндж) регрессия

- В лассо улучшение от ошибки 5 до ошибки 4 взвешивается так же, как улучшение от 4 до 3, а также от 3 до 2, от 2 до 1 и от 1 до 0. Следовательно, больше коэффициентов достигает нуля и устраняется больше признаков.
- В гребневой регрессии улучшение от ошибки 5 до ошибки 4 вычисляется как $5^2 - 4^2 = 9$, тогда как улучшение от 4 до 3 взвешивается только как 7. Постепенно вознаграждение за улучшение уменьшается; следовательно, устраняется меньше признаков.

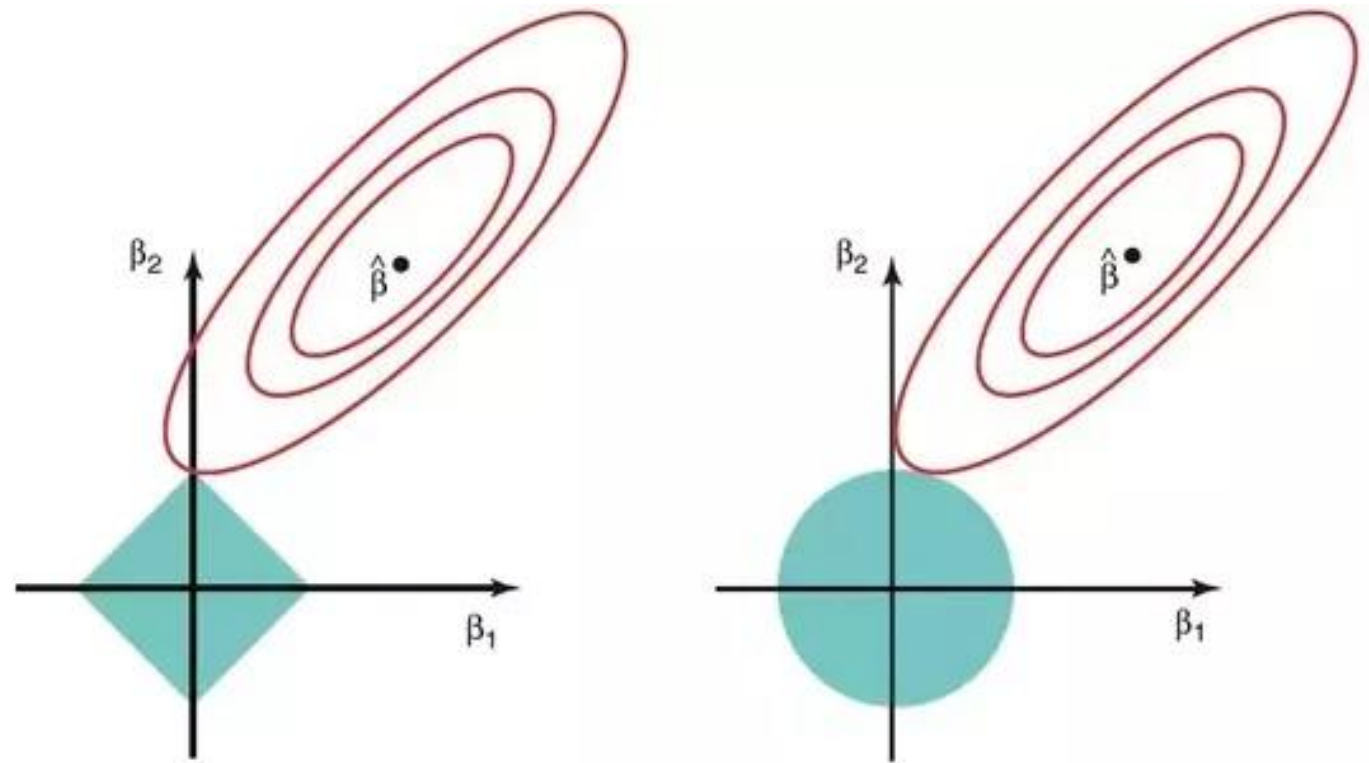


FIGURE 6.7. Contours of the error and constraint functions for the lasso (left) and ridge regression (right). The solid blue areas are the constraint regions, $|\beta_1| + |\beta_2| \leq s$ and $\beta_1^2 + \beta_2^2 \leq s$, while the red ellipses are the contours of the RSS.

Гребневая (Риндж) регрессия

```
from sklearn.linear_model import RidgeCV
```

```
model = Ridge()
```

```
model.fit(X_train, y_train)
```

- Тьюториал (короткий) по всем трем регрессиям

<https://www.kaggle.com/code/vignesh1609/ridge-lasso-linear-regression-models>

- Lasso VS Ridge <https://www.kaggle.com/code/rrrohit/regression-ridge-or-lasso-multicollinearity>